

Fundamentos de Arquitectura de Computadores

Práctica 0

Introducción a RISC-V y al simulador Ripes

La ISA RV32IM · El entorno de trabajo · Primer programa en ensamblador

Grado en Ingeniería Informática · Universidad de Jaén

Objetivos de la práctica

Esta práctica prepara el entorno y sienta las bases para las prácticas evaluables posteriores.

- ◆ Conocer qué es la **ISA de RISC-V** y por qué la usamos en la asignatura.
- ◆ Entender la diferencia entre un **simulador** y el hardware o un emulador, y por qué trabajamos con simulador en las primeras prácticas.
- ◆ Descargar e instalar **Ripes**, el simulador que usaremos.
- ◆ Familiarizarnos con la **interfaz** de Ripes: editor, vista del procesador y vista de memoria.
- ◆ Escribir, ensamblar y ejecutar un **primer programa**, examinando registros y memoria.

¿Qué es una ISA?

Se denomina **ISA** (*Instruction Set Architecture*) a la interfaz que determina cómo, mediante software, se controla el hardware de un microprocesador.

Forman parte de la ISA aspectos como:

- ◆ El **banco de registros** visibles a los programas.
- ◆ Los **modos de direccionamiento** para acceder a memoria.
- ◆ El **conjunto de instrucciones** disponible.

RV32IM

La ISA de RISC-V que usaremos: **RISC-V 32 bits Integer and Multiplication** (versión 2.0).

- Ancho de palabra y de instrucción: **32 bits**.
- Aritmética entera, incluyendo **multiplicación y división**.

La ISA permite escribir programas para cualquier CPU que la implemente

Ejemplo — una palabra de 32 bits. Cada dígito hexadecimal representa **4 bits**, luego 32 bits = **8 dígitos hexadecimales** = 4 bytes:

0x0000 0000 = 00000000 00000000 00000000 00000000

0xFFFF FFFF = 11111111 11111111 11111111 11111111

Son la primera y la última dirección de memoria del espacio de 32 bits.

¿Por qué RISC-V?

RISC-V es la **quinta evolución** del diseño RISC-I surgido en la Universidad de Berkeley.

- ◆ **Utilidad futura:** se estima que en 2025 habrá más de **60 000 millones** de núcleos RISC-V en funcionamiento.
- ◆ **ISA abierta:** gestionada por RISC-V International; no hay que pagar licencias para fabricar procesadores con ella.
- ◆ **Modular:** se parte de un núcleo (p. ej. RV32I) y se añaden extensiones (multiplicación, punto flotante, vectorial...).
- ◆ **Objetivo estratégico europeo** (proyecto DARE) para la autonomía tecnológica.

> **60 000 M**

núcleos RISC-V estimados en 2025

Abierta

ISA sin licencias, especificación pública

Modular

núcleo + extensiones ratificadas

RISC y CISC

RISC · Reduced Instruction Set

- ◆ Pocas instrucciones, para operaciones básicas.
- ◆ Electrónica más simple y fácil de optimizar.
- ◆ Longitud de instrucción fija (32 bits en RV32IM).
- ◆ Productos más pequeños y de menor consumo (móviles, embebidos).

CISC · Complex Instruction Set

- ◆ Instrucciones complejas que hacen mucho trabajo.
- ◆ Implementación más costosa (espacio, consumo, calor).
- ◆ Longitud de instrucción variable.
- ◆ Muchos CISC actuales tienen internamente un núcleo RISC (p. ej. Intel P6).

Con compiladores modernos capaces de optimizar el código, gran parte de las ISA actuales son **RISC**, ya que permite fabricar procesadores más sencillos, pequeños y de menor consumo.

Entorno de trabajo: ¿simulador o hardware?

Procesador físico

Metodología real. Pero ejecuta muchos procesos del S.O., por lo que es difícil medir con exactitud los ciclos o ver el estado interno del cauce.

Emulador HW (QEMU)

Entorno muy parecido al hardware real: permite instalar un S.O. y usar herramientas estándar. Mismas ventajas e inconvenientes que el físico.

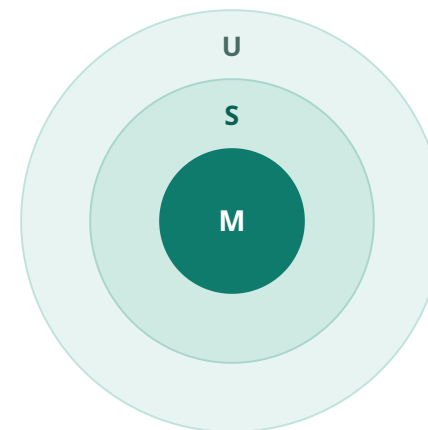
Simulador ✓

Ejecuta **solo tu código**. Permite observar con total exactitud registros, memoria, caché y cada etapa del cauce. Ideal para **aprender**.

Modos de operación del procesador

La arquitectura RISC-V implementa varios **modos de operación** con diferentes privilegios (niveles o anillos de protección). RISC-V define tres:

- ◆ **M-mode** (Machine): el único indispensable en todo núcleo RISC-V; máximo privilegio.
- ◆ **S-mode** (Supervisor): pensado para los sistemas operativos.
- ◆ **U-mode** (User): mínimo privilegio; ejecuta las aplicaciones de usuario.



No es imprescindible conocer los niveles de privilegio. El **simulador implementa únicamente la ISA del modo usuario**; con hardware real o QEMU existirían los tres.

La herramienta Ripes

De entre los simuladores RISC-V disponibles usaremos **Ripes**, un entorno que reúne en una sola herramienta:

Editor

Escritura del código ensamblador.

Ensamblador

Traduce a código máquina RV32IM.

Visor del cauce

Muestra la ejecución de las instrucciones.

Visor de memoria

Inspecciona direcciones y su contenido.

Registros

Estado del banco de 32 registros.

Emulación de E/S

Dispositivos de entrada/salida (Práctica 3).

Ripes permite editar y ejecutar programas para las ISA **RV32IM** y **RV64IM**, examinando registros, memoria y dispositivos de E/S.

Descarga del simulador

Ripes se descarga del apartado **Releases** del repositorio oficial en GitHub. Versión actual: **v2.2.6**.

↓ **Descargar Ripes v2.2.6 (Windows)**

[Ver todas las versiones \(página Releases\)](#) ↗

- ◆ **En el laboratorio no hay que descargar nada:** Ripes ya está instalado. Para tu ordenador, tienes el archivo de Windows en la **plataforma de docencia virtual** (versión validada para el aula) o en GitHub.

¿Qué archivo descargar?

Elige según tu sistema operativo:

Sistema	Archivo a descargar
Windows	Ripes-v2.2.6-win-x86_64.zip
macOS	Ripes-v2.2.6-mac-x86_64.zip
GNU/Linux	Ripes-v2.2.6-linux-x86_64.AppImage

El archivo de Windows ocupa unos 15 MB.

Instalación en Windows, paso a paso



En el laboratorio, Ripes ya está instalado

Estos pasos son solo para instalarlo en tu propio ordenador.

- ◆ 1. Localiza el archivo descargado `Ripes-v2.2.6-win-x86_64.zip` (normalmente en la carpeta **Descargas**).
- ◆ 2. Haz **clic derecho** → «**Extraer todo...**» y elige una carpeta, por ejemplo `Documentos\Ripes`.
- ◆ 3. Abre la carpeta extraída y haz **doble clic en** `Ripes.exe`.
- ◆ 4. Si aparece «Windows protegió su PC», pulsa «**Más información**» → «**Ejecutar de todas formas**».

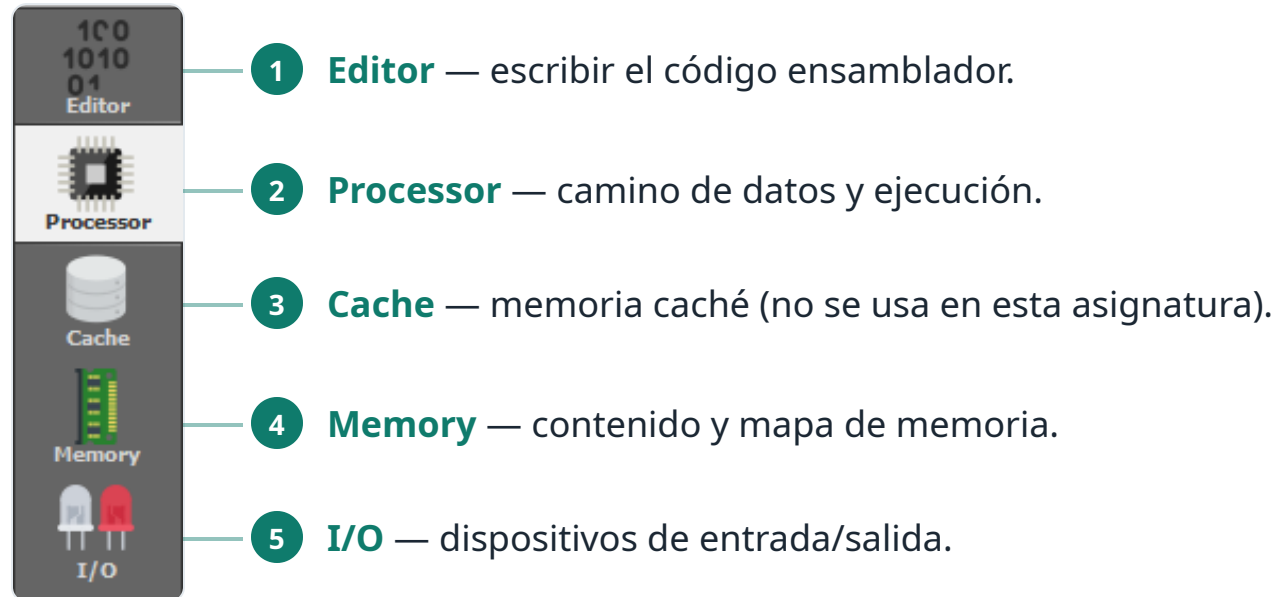
Si sale el error `msvcp140.dll` (falta la biblioteca de Visual C++): instala el paquete **Visual C++ Redistributable** desde el sitio oficial de Microsoft (descarga id=48145) y vuelve a abrir Ripes.

Otros sistemas operativos

- ◆ **GNU/Linux:** clic derecho en el AppImage → Propiedades → permitir ejecución (o `chmod a+x`) y ejecútalo.
- ◆ **macOS:** descomprime el `.zip` y ejecuta la aplicación.

La barra lateral: vistas de Ripes

Ripes se organiza en **vistas** que se eligen en la **barra lateral izquierda**.
Cada icono abre una vista:



La barra de herramientas

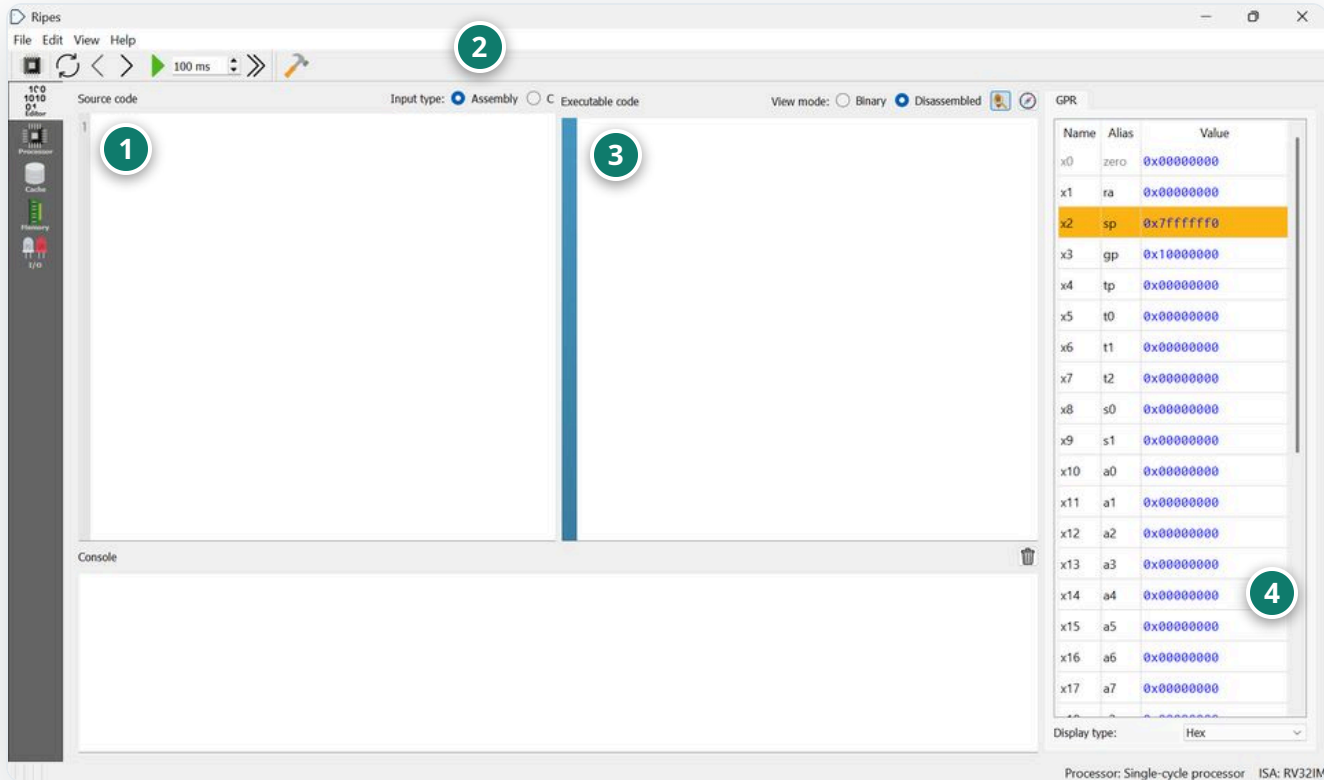
Desde la barra superior se controla la **ejecución** del programa:



- 1 **Seleccionar** el procesador.
- 2 **Reiniciar** la simulación.
- 3 Retroceder una instrucción.
- 4 Ejecutar la **siguiente instrucción** (paso a paso).
- 5 **Ejecutar** el programa.
- 6 **Velocidad** de la ejecución automática.
- 7 Ejecutar **hasta el final**.

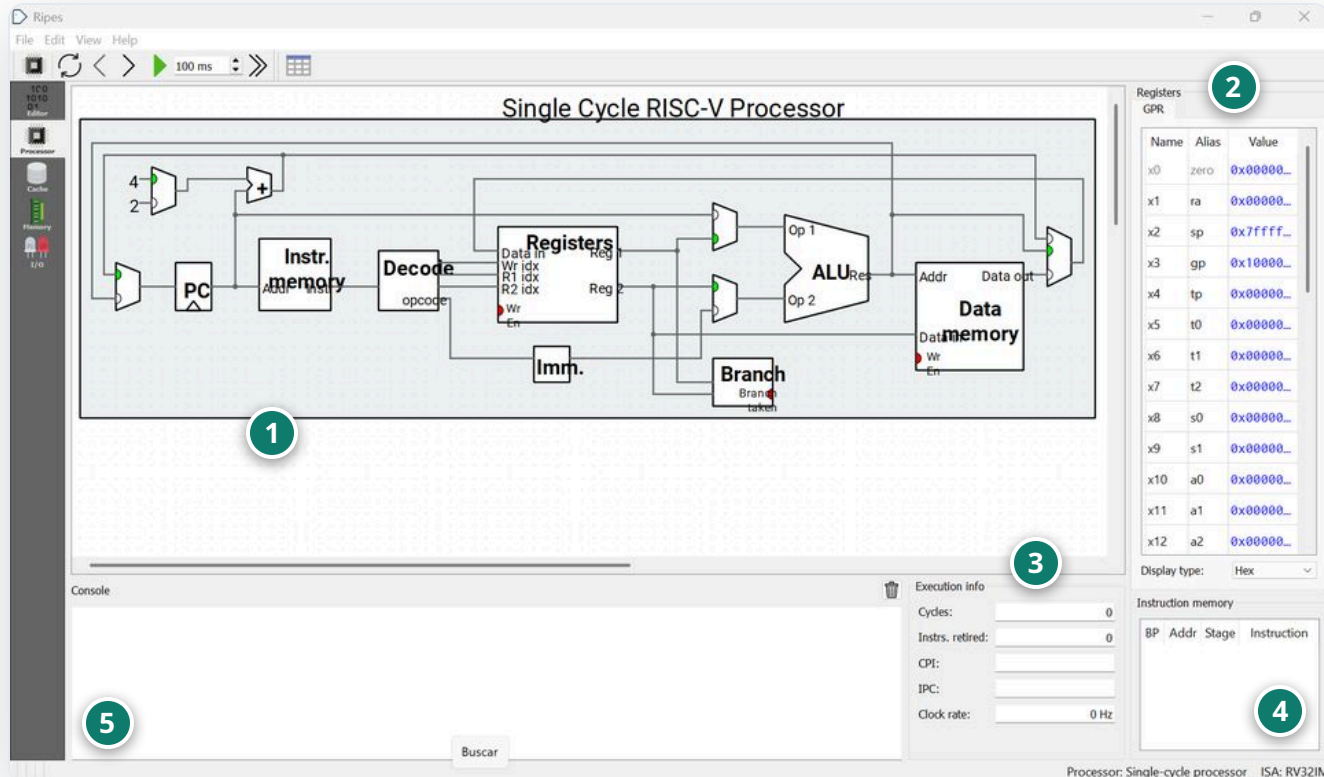
El último icono de la derecha ofrece opciones propias de cada vista.

Vista Editor



- 1 **Source code**: aquí escribes tu programa en ensamblador.
- 2 **Input type**: selecciona **Assembly** (o C).
- 3 **Executable code**: el código máquina generado, en **Binary** o **Disassembled**.
- 4 **Registers** (derecha): estado de los 32 registros; el que cambia se resalta.

Vista Processor



- 1 **Camino de datos** del procesador de **ciclo único**: PC, memoria de instrucciones, decodificación, registros, ALU y memoria de datos.
- 2 **Registers**: valor de cada registro.
- 3 **Execution info**: ciclos, instrucciones retiradas, **CPI**, **IPC** y frecuencia.
- 4 **Instruction memory**: instrucciones cargadas y su dirección.
- 5 **Console**: mensajes y salida del programa.

Vista Memory

The screenshot shows the Ripes Memory viewer interface. It features a menu bar (File, Edit, View, Help), a toolbar with navigation and execution controls, and a sidebar with system components (CPU, Cache, Memory). The main area is divided into two panes: the Memory viewer on the left and the Memory map on the right. The Memory viewer displays a table of memory addresses, words, and bytes. The Memory map displays a table of memory sections (text, data, bss) with their sizes and ranges. At the bottom, there are controls for display type, going to a register, and going to a section. The status bar at the bottom right indicates the processor is a Single-cycle processor and the ISA is RV32IM.

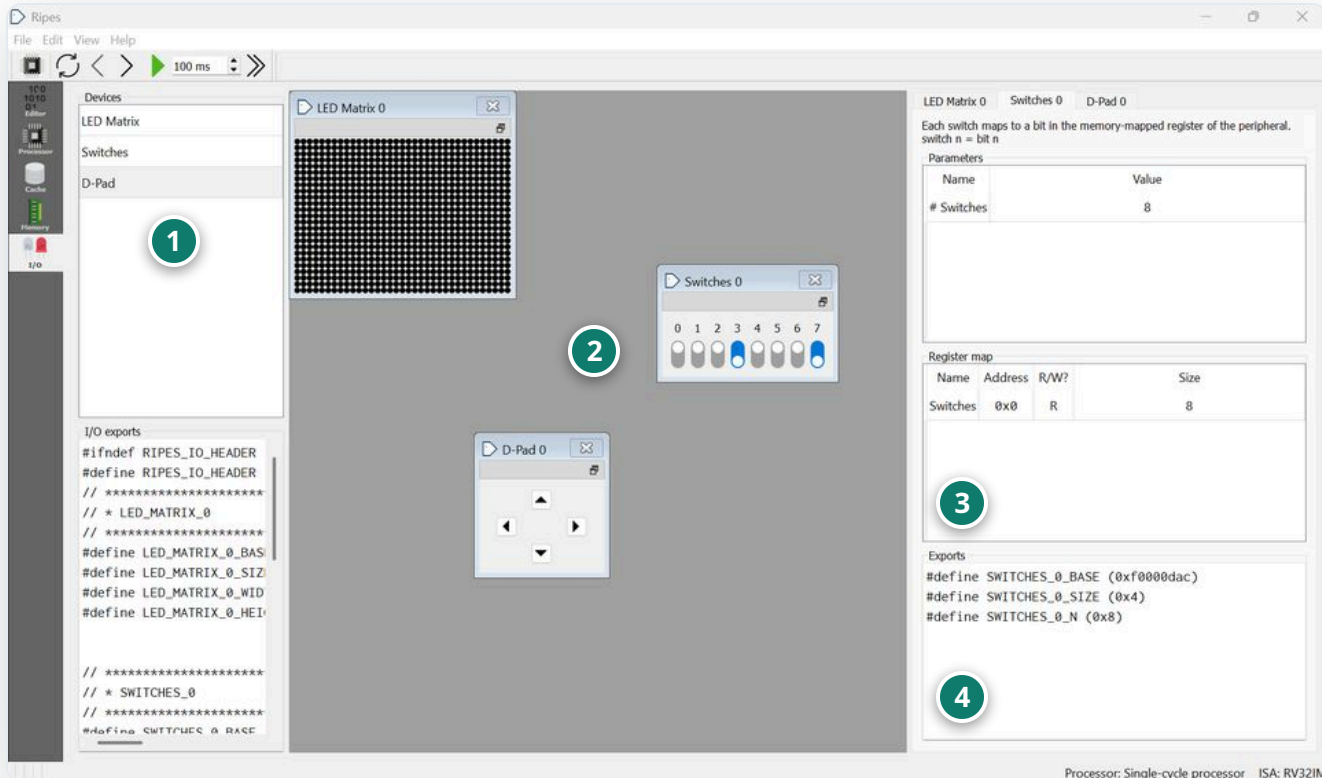
Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
0x00000028	X	X	X	X	X
0x00000024	X	X	X	X	X
0x00000020	X	X	X	X	X
0x0000001c	X	X	X	X	X
0x00000018	X	X	X	X	X
0x00000014	X	X	X	X	X
0x00000010	X	X	X	X	X
0x0000000c	X	X	X	X	X
0x00000008	X	X	X	X	X
0x00000004	X	X	X	X	X
0x00000000	0x00000000	0x00	0x00	0x00	0x00
...	...	...	...	...	...

Name	Size	Range
.text	0	0x00000000 - 0xffffffff
.data	0	0x10000000 - 0xffffffff
.bss	0	0x11000000 - 0xffffffff

Display type: Hex Go to register: Go to section: Processor: Single-cycle processor ISA: RV32IM

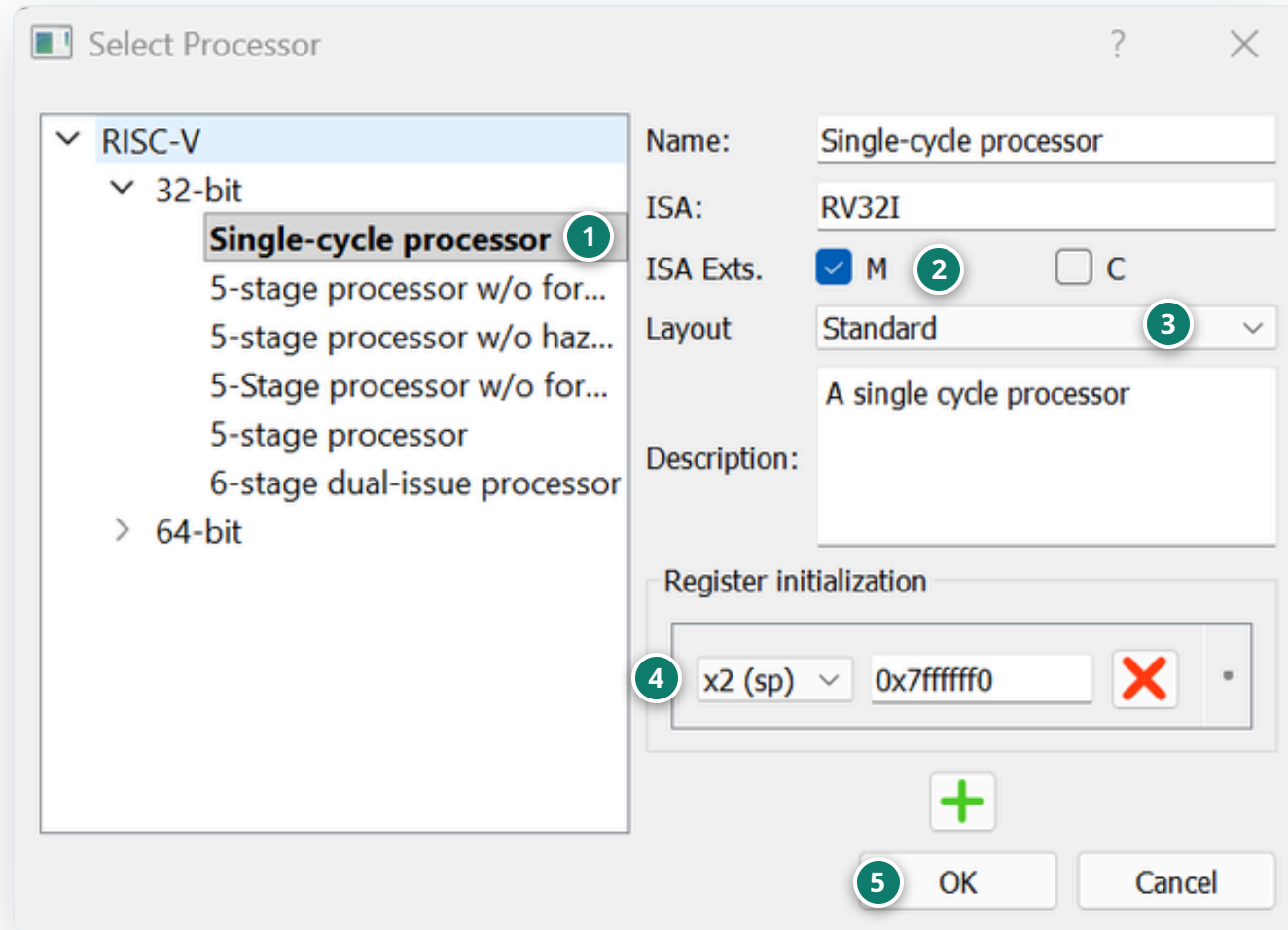
- 1 **Memory viewer:** dirección y contenido, por palabra (Word) y por bytes (Byte 0–3).
- 2 **Memory map:** secciones `.text`, `.data` y `.bss` con sus rangos.
- 3 **Go to section / register:** salta a una sección o a la dirección de un registro.
- 4 **Display type:** formato de visualización (Hex, decimal...).

Vista I/O — entrada/salida



- 1 **Devices:** lista de dispositivos disponibles (a la izquierda).
- 2 Los dispositivos aparecen en el **área central:** LED Matrix, Switches y D-Pad.
- 3 **Register map:** cada dispositivo tiene una **dirección base** y un tamaño (E/S mapeada).
- 4 **Exports:** los `#define` con la dirección base (p. ej. `SWITCHES_0_BASE`).

Configurar el procesador: ciclo único



Se abre con el primer botón de la barra de herramientas (icono del procesador)

- 1 Elige **RISC-V** > **32-bit** > **Single-cycle processor**: cada etapa ocupa 1 ciclo.
- 2 ISA **RV32I** con la extensión **M** marcada → trabajamos con **RV32IM**.
- 3 **Layout**: Standard.
- 4 **Inicialización**: el puntero de pila **x2 (sp)** = 0x7ffffff0.
- 5 Pulsa **OK**.

Nuestro primer programa

Escribimos en el **editor** un programa que carga dos números de memoria, los suma y almacena el resultado:

- ◆ Las directivas `.data`, `.bss` y `.text` marcan el inicio de las secciones de datos inicializados, datos sin inicializar y código.
- ◆ `.word` reserva espacio para un entero de 32 bits.
- ◆ La ejecución termina con `ecall` (función 10, *exit*).

 Ejecuta el programa con **F8** y observa el resultado en la lista de registros y en la consola.

```
.data                # Segmento de datos
n1: .word 7          # dos valores
n2: .word 3          # enteros

.bss                # Datos sin inicializar
res: .word 0

.text              # Segmento de código
lw  a1, n1         # a1 = (n1)
lw  a2, n2         # a2 = (n2)
add a0, a1, a2     # a0 = a1 + a2
la  t0, res        # t0 -> res
sw  a0, 0, t0      # (res) = a0
addi a7, zero, 10
ecall              # función 10 (exit)
```

El resultado en memoria

Tras ejecutar, abre la vista **Memory** para ver los datos. Ajusta lo marcado en **amarillo**: pon **Display type** en **Signed**, elige la sección con **Go to section** y localiza las **direcciones** que contienen los datos.

0x10000028	X	X	X	X	X
0x10000024	X	X	X	X	X
0x10000020	X	X	X	X	X
0x1000001c	X	X	X	X	X
0x10000018	X	X	X	X	X
0x10000014	X	X	X	X	X
0x10000010	X	X	X	X	X
0x1000000c	X	X	X	X	X
0x10000008	0	0	0	0	0
0x10000004	3	3	0	0	0
0x10000000	7	7	0	0	0
0x0ffffffc	X	X	X	X	X
0x0ffffff8	X	X	X	X	X
0x0ffffff4	X	X	X	X	X
0x0fffff0	X	X	X	X	X

Display type: **Signed** Go to register: Select Go to section: **Select**

.data · n1 = 7 en 0x10000000, n2 = 3 en 0x10000004

0x1100001c	X	X	X	X	X
0x11000018	X	X	X	X	X
0x11000014	0	0	0	0	0
0x11000010	X	X	X	X	X
0x1100000c	X	X	X	X	X
0x11000008	X	X	X	X	X
0x11000004	X	X	X	X	X
0x11000000	10	10	0	0	0
0x10fffffc	X	X	X	X	X
0x10fffff8	X	X	X	X	X
0x10fffff4	X	X	X	X	X
0x10fffff0	X	X	X	X	X
0x10ffffec	X	X	X	X	X
0x10ffffe8	X	X	X	X	X
0x10ffffe4	X	X	X	X	X

Display type: **Signed** Go to register: Select Go to section: **Select**

.bss · el resultado res = 10 en 0x11000000

Ejecutar, examinar y guardar

Ejecutar

Tras ensamblar sin errores, ejecuta el programa (completo o paso a paso) con los controles de la barra de herramientas.

Examinar registros y memoria

En la lista de **registros**, `a0` contendrá el resultado y `t0` apuntará a `res`. Abre la vista **Memory** para ver las direcciones y su contenido (Hex, decimal...) y el mapa de memoria.

Guardar y cargar

- ◆ Guardar y cargar código está en el menú **File**.
- ◆ Los programas se guardan con extensión `.s`
- ◆ Será la vía para **entregar los ejercicios** en la plataforma de docencia virtual.
- ◆ **File** → **Load Example** → **Assembly**: programas de ejemplo del propio software.

 Experimenta cambiando entre secciones de memoria y formatos (Hex, decimal...).



Universidad
de Jaén

YA TIENES EL ENTORNO LISTO

¿Qué viene después?

En las siguientes prácticas evaluables aplicaremos lo aprendido:

- ◆ **Práctica 1** — Introducción a la ISA: registros, instrucciones y direccionamiento.
- ◆ **Práctica 2** — Bucles, condicionales, subrutinas y pila.
- ◆ **Práctica 3** — Operaciones de entrada/salida.
- ◆ **Práctica 4** — Evaluación de prestaciones (benchmarking).

Recursos: repositorio github.com/mortbopet/Ripes